

DEV_IMPL_GUIDE - SUB-WF-RESUME-01-FLOW-WIK

1. Purpose

This guide explains how to implement and test the WIK resume BPMN as the flow executes at runtime.

Use it with:

- DD_SUB-WF-RESUME-01-FLOW-WIK-v1.2.md
- DD_SUB-WF-RESUME-01-Subscription_Resume-v1.2.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md

This guide is specific to the BPMN process file:

sub-resume-wik.bpmn

2. Runtime Actors

Runtime component	Role in this flow
subscription-workflow-api pod	Receives WIK resume request and calls the shared operation framework.
SUB-LM service/module	Provides authoritative subscription summary and status mutation APIs.
Pause/history persistence	Provides and closes the open pause-history row.
Drools	Evaluates rules.subscription.resume.wik.
Camunda	Runs process sub-resume-wik.
Worker apps	Execute external task topics from the BPMN. One app can handle many topics.
BIL-01	Evaluates RESUME_INTENT or RESUME_NP_INTENT.
BIL-04	Calls dunning resume after debt settlement.
FUL-03	Reactivates service.
Message correlators	Correlate payment and fulfillment messages back to Camunda.
NOT-01	Sends customer notification after success.
FOUNDATION_SCHEDULER	Calls scheduled auto-resume when a pause row is due.

3. Before The Flow Can Run

Confirm these objects exist before deploying or testing the BPMN:

Object	Required value
BPMN process key	sub-resume-wik
Operation config	(operator_code=WIK, operation_kind=RESUME) -> sub-resume-wik
Resume config	WIK row in subscription_resume_config
Pause-history table	Open rows from PAUSE or SUSPEND-NP exist.
Drools package	rules.subscription.resume.wik
Worker subscriptions	Required worker topics from the WIK DD
Message correlators	payment-confirmed, fulfillment-completed, fulfillment-failed
BIL-01 intents	RESUME_INTENT, RESUME_NP_INTENT

4. Step 1 - Caller Sends Request

Public endpoint:

```
POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/resume HTTP/1.1
Authorization: Bearer <bil04-service-token>
Idempotency-Key: resume-sub_01HZ_KAMAU-bil04-payment-cleared-20260622
Content-Type: application/json
```

```
{
  "operatorCode": "WIK",
  "resumeTrigger": "DUNNING_RESUME",
  "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
  "dunningCycleReference": "bil04-resume-2026-06-22",
  "notes": "Customer paid INV_01HZ KES 3,500 via M-Pesa",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-resume-2026-06-22"
}
```

Actor user and role must come from the authenticated service token. For this example:

```
{
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL"
}
```

5. Step 2 - API Pod Validates Shape And Idempotency

The subscription-workflow-api pod validates:

Field	Validation
operatorCode	Required and must be WIK for this BPMN.
resumeTrigger	One of the four parent DD trigger values.
resumeReasonCode	Required.
dunningCycleReference	Required when trigger is DUNNING_RESUME.
adminForceResumeReason	Required when admin force-resuming SUSPEND_NP.

Idempotency behavior:

- First request inserts the operation and starts BPMN.
- Same Idempotency-Key with same request hash returns the existing operation response.
- Same Idempotency-Key with a different request hash must return 409 IDEMPOTENCY_KEY_REUSED.

Implementation note:

Do not insert a new operation before checking existing key and request hash. Use a transaction or unique constraint on the idempotency key plus operator/operation scope so concurrent duplicate requests cannot create two operations.

6. Step 3 - API Loads Required Data

6.1 Subscription Summary

The API gets current subscription state from SUB-LM.

Same deployable:

```
SubscriptionReadService.getSummary(subscriptionId)
```

Split services:

```
GET /internal/subscriptions/sub_01HZ_KAMAU_FIBER/summary
Authorization: Bearer <subscription-workflow-service-token>
```

Expected response:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "statusCode": "SUSPENDED",
  "packageRef": "pkg_FIBER_HOME_50M",
  "homepassId": "hp_01HZ_KAREN_017",
  "activeRestrictions": []
}
```

Do not use cache for current subscription status. Resume is a state-changing workflow and must use the authoritative SUB-LM read path.

6.2 Resume Config

Read from subscription workflow persistence:

```
SELECT *
FROM subscription_resume_config
WHERE operator_code = 'WIK';
```

Required sample:

```
{
  "operatorCode": "WIK",
  "customerSelfServiceEnabled": true,
  "reconnectionFeeOnCustomerResume": false,
  "dunningResumeVerifyDebtCleared": true,
  "customerNotificationEnabled": true
}
```

6.3 Open Pause-History Row

Read the open row:

```
SELECT *
FROM subscription_pause_history
WHERE subscription_id = :subscriptionId
AND actual_resume_at IS NULL;
```

Required sample for dunning resume:

```
{
  "pauseId": "pause_01HZ_KAMAU_SUSPNP",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "originIntent": "SUSPEND_NP",
  "systemManaged": true,
  "suspendedAt": "2026-06-15T02:00:18Z",
  "actualResumeAt": null,
  "dunningCycleReference": "bil04-dunning-wave-2026-06-15-L3",
  "outstandingDebtAmount": 3500.00,
  "outstandingDebtCurrency": "KES"
}
```

If no row exists, return:

```
409 Conflict

{
  "errorCode": "NO_OPEN_PAUSE_HISTORY",
  "message": "Subscription is suspended but no open pause-history row exists."
}
```

7. Step 4 - API Evaluates Drools Before Starting BPMN

Call Drools package:

```
rules.subscription.resume.wik
```

Facts:

```
{
  "operatorCode": "WIK",
  "subscription": {
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "statusCode": "SUSPENDED"
  },
  "request": {
    "resumeTrigger": "DUNNING_RESUME",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "dunningCycleReference": "bil04-resume-2026-06-22"
  },
  "openPause": {
    "pauseId": "pause_01HZ_KAMAU_SUSPNP",
    "originIntent": "SUSPEND_NP",
    "systemManaged": true,
    "resumeScheduledAt": null,
    "actualResumeAt": null
  },
  "config": {
    "customerSelfServiceEnabled": true,
    "reconnectionFeeOnCustomerResume": false,
    "dunningResumeVerifyDebtCleared": true
  },
  "actor": {
    "userId": "bil04-svc-account",
    "role": "BILLING_INTERNAL"
  }
}
```

Expected decision:

```
{
  "eligible": true,
  "originGatingAllowed": true,
  "billingIntentKind": "RESUME_NP_INTENT",
  "requiresDebtClearedCheck": true,
  "requiresReconnectionFeePayment": false,
  "reconnectionFeeApplicable": false
}
```

If Drools rejects, return the mapped parent-DD error and do not start Camunda.

8. Drools Rules To Configure

8.1 Suspended Subscription

```
rule "WIK resume requires suspended subscription"
when
  $req: ResumeRequestFact()
  $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "SUSPENDED")
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("INVALID_STATE_FOR_RESUME")
  }
end
```

8.2 Open Pause Row

```
rule "WIK resume requires open pause history"
when
  $req: ResumeRequestFact(eligible == null)
  not (PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null))
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("NO_OPEN_PAUSE_HISTORY")
  }
end
```

8.3 Trigger Role Gating

```
rule "WIK customer resume allowed roles"
when
  $req: ResumeRequestFact(
    resumeTrigger == "CUSTOMER_REQUESTED_RESUME",
    eligible == null,
    initiatingActorRole not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR")
  )
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
  }
end

rule "WIK admin force resume allowed roles"
when
  $req: ResumeRequestFact(
    resumeTrigger == "ADMIN_FORCE_RESUME",
    eligible == null,
    initiatingActorRole not in ("SUBSCRIPTION_ADMIN", "SUBSCRIPTION_OPERATOR")
  )
then
  modify($req) {
    setEligible(false);
    setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
  }
end

rule "WIK dunning resume requires billing internal"
when
  $req: ResumeRequestFact(
    resumeTrigger == "DUNNING_RESUME",
    eligible == null,
    initiatingActorRole != "BILLING_INTERNAL"
  )
then
  modify($req) {
```

```

        setEligible(false);
        setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
    }
end

rule "WIK scheduled auto resume requires scheduler internal"
when
    $req: ResumeRequestFact(
        resumeTrigger == "SCHEDULED_AUTO_RESUME",
        eligible == null,
        initiatingActorRole != "SCHEDULER_INTERNAL"
    )
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER")
    }
end

```

8.4 Origin-Intent Gating

```

rule "WIK customer cannot resume admin pause"
when
    $req: ResumeRequestFact(resumeTrigger == "CUSTOMER_REQUESTED_RESUME", originGatingAllowed == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent == "ADMIN_PAUSE")
then
    modify($req) {
        setOriginGatingAllowed(false);
        setOriginGatingReasonCode("ADMIN_PAUSE_BLOCKS_CUSTOMER_RESUME")
    }
end

rule "WIK customer cannot resume non payment suspension"
when
    $req: ResumeRequestFact(resumeTrigger == "CUSTOMER_REQUESTED_RESUME", originGatingAllowed == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent == "SUSPEND_NP")
then
    modify($req) {
        setOriginGatingAllowed(false);
        setOriginGatingReasonCode("SYSTEM_MANAGED_SUSPENSION")
    }
end

rule "WIK dunning resume only resumes non payment suspension"
when
    $req: ResumeRequestFact(resumeTrigger == "DUNNING_RESUME", originGatingAllowed == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent != "SUSPEND_NP")
then
    modify($req) {
        setOriginGatingAllowed(false);
        setOriginGatingReasonCode("NOT_A_DUNNING_SUSPENSION")
    }
end

rule "WIK scheduled auto resume blocks non payment suspension"
when
    $req: ResumeRequestFact(resumeTrigger == "SCHEDULED_AUTO_RESUME", originGatingAllowed == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, originIntent == "SUSPEND_NP")
then
    modify($req) {
        setOriginGatingAllowed(false);
        setOriginGatingReasonCode("NOT_A_SCHEDULED_RESUME")
    }
end

rule "WIK scheduled auto resume requires resume scheduled at"
when
    $req: ResumeRequestFact(resumeTrigger == "SCHEDULED_AUTO_RESUME", eligible == null)
    $pause: PauseHistoryFact(subscriptionId == $req.subscriptionId, actualResumeAt == null, resumeScheduledAt == null)
then
    modify($req) {
        setEligible(false);
        setEligibilityReasonCode("NO_SCHEDULED_RESUME_TIME")
    }
end

```

8.5 Billing Intent And Fee

```

rule "WIK dunning resume uses resume non payment intent"
when
    $req: ResumeRequestFact(resumeTrigger == "DUNNING_RESUME")
then

```

```

    modify($req) {
        setBillingIntentKind("RESUME_NP_INTENT");
        setRequiresDebtClearedCheck(true);
        setRequiresReconnectionFeePayment(false)
    }
end

rule "WIK reconnection fee applies only to configured customer resume"
when
    $req: ResumeRequestFact(resumeTrigger == "CUSTOMER_REQUESTED_RESUME")
    $cfg: OperatorConfigFact(reconnectionFeeOnCustomerResume == true)
then
    modify($req) {
        setBillingIntentKind("RESUME_INTENT");
        setRequiresReconnectionFeePayment(true);
        setReconnectionFeeApplicable(true)
    }
end

rule "WIK resume billing default"
salience -100
when
    $req: ResumeRequestFact(billingIntentKind == null)
then
    modify($req) {
        setBillingIntentKind("RESUME_INTENT");
        setRequiresDebtClearedCheck(false);
        setRequiresReconnectionFeePayment(false);
        setReconnectionFeeApplicable(false)
    }
end

```

9. Step 5 - API Starts Shared Operation

After validation and Drools approval, the API asks the shared SUB-WF framework to create the operation and start Camunda.

Same deployable:

```
OperationStartService.start(StartOperationCommand command)
```

Split services:

```

POST /internal/subscription-operations/start
Authorization: Bearer <subscription-workflow-service-token>
Content-Type: application/json

```

The payload below is the internal command passed to the framework start contract. It is not the public resume API request.

```

{
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operationKind": "RESUME",
  "idempotencyKey": "resume-sub_01HZ_KAMAU-bil04-payment-cleared-20260622",
  "initiatingActorUserId": "bil04-svc-account",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "bil04-resume-2026-06-22",
  "bpmnVariables": {
    "customerId": "cust_01HZ_KAMAU",
    "resumeTrigger": "DUNNING_RESUME",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "openPauseHistoryRowId": "pause_01HZ_KAMAU_SUSPNP",
    "openPauseOriginIntent": "SUSPEND_NP",
    "billingIntentKind": "RESUME_NP_INTENT",
    "requiresDebtClearedCheck": true,
    "requiresReconnectionFeePayment": false,
    "dunningCycleReference": "bil04-resume-2026-06-22",
    "notes": "Customer paid INV_01HZ KES 3,500 via M-Pesa"
  }
}

```

The framework:

1. Inserts subscription_operation.
2. Resolves (WIK, RESUME) to sub-resume-wik.
3. Starts Camunda process sub-resume-wik.
4. Stores the Camunda process instance id against the operation.

API response:

```
{
  "operationId": "subop_01HZ_KAMAU_RESUME",
  "operationKind": "RESUME",
  "bpmnProcessKey": "sub-resume-wik",
  "currentState": "INITIATED"
}
```

10. Step 6 - BPMN Runs drools-decide

The BPMN should run drools-decide again as a guard because state may have changed between API validation and Camunda execution.

Service task:

```
<bpmn:serviceTask id="Task_drools_validate"
  name="Validate resume policy"
  camunda:type="external"
  camunda:topic="drools-decide" />
```

Input variables:

```
{
  "kjarPackage": "rules.subscription.resume.wik",
  "requestedDecisions": [
    "validate-preconditions",
    "validate-origin-intent-gating",
    "compute-reconnection-fee-applicability"
  ],
  "factsJson": "{...fresh subscription state, open pause row, config, actor...}"
}
```

If rejected:

```
kafka-emit SubscriptionResumeRejected
subscription-operation-finalize FAILED
```

11. Step 7 - BPMN Moves Subscription To Pending Resume

Service task topic:

```
sub-lm-put-pending-status
```

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "toStatus": "PENDING_RESUME",
  "currentTransitionType": "RESUME",
  "currentTransitionReasonCode": "DUNNING_RESUME"
}
```

Worker output:

```
{
  "priorStatusSnapshot": {
    "statusCode": "SUSPENDED"
  },
  "lmEventId": "lmevt_01HZ_PENDING_RESUME"
}
```

Compensation later uses the priorStatusSnapshot if the BPMN must revert.

12. Step 8 - BPMN Computes Billing Context

Service task topic:

```
sub-lm-compute-cycle-window
```

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "referenceDate": "2026-06-22T10:00:00Z"
}
```

Worker output:

```
{
  "cycleWindow": {
    "cycleStart": "2026-06-01",
```

```

        "cycleEnd": "2026-06-30",
        "cycleAnchorDay": 1
    }
}

```

13. Step 9 - BPMN Sends Billing Intent

Service task topic:

```
bil01-emit-intent
```

Worker input for dunning resume:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intentKind": "RESUME_NP_INTENT",
  "intentContext": {
    "resumeTrigger": "DUNNING_RESUME",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "openPauseHistoryRowId": "pause_01HZ_KAMAU_SUSPNP",
    "openPauseOriginIntent": "SUSPEND_NP",
    "pausedSince": "2026-06-15T02:00:18Z",
    "pauseDurationDays": 7,
    "dunningCycleReference": "bil04-resume-2026-06-22",
    "verifyDebtCleared": true
  },
  "cycleWindow": {
    "cycleStart": "2026-06-01",
    "cycleEnd": "2026-06-30"
  }
}

```

Expected output:

```

{
  "intentDecisionId": "bidec_01HZ_KAMAU_RESUME",
  "debtClearedConfirmation": true,
  "lineItems": [],
  "totals": {
    "net": 0,
    "currency": "KES"
  }
}

```

If debtClearedConfirmation=false, route to failure with DEBT_NOT_CLEARED.

14. Step 10 - BPMN Evaluates Optional Payment Gateway

Gateway condition:

```
${requiresReconnectionFeePayment == true}
```

For WIK v1.2 default config:

```
requiresReconnectionFeePayment == false
```

So the BPMN skips payment and continues to FUL-03.

If future WIK config enables a reconnection fee, the BPMN runs:

```

bil01-charge-pre-authorize
wait payment-confirmed with boundary timer P7D
bil01-confirm-charge

```

Payment timeout route:

```

bil01-revert-pending-charge
sub-lm-revert-to-prior
kafka-emit SubscriptionResumeRejected
subscription-operation-finalize FAILED

```

These names are worker topics or BPMN message names, not shell commands.

15. Step 11 - BPMN Calls FUL-03 To Reactivate Service

Service task topic:

```
ful-call-activate
```

Worker input:


```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intent": "RESUME_SERVICE",
  "policy": "RESUME_FROM_SUSPENSION",
  "fulfillmentCorrelationId": "subop_01HZ_KAMAU_RESUME:resume"
}
```

The worker calls FUL-03. FUL-03 may return quickly or asynchronously depending on network adapters.

The BPMN then waits for one of:

```
fulfillment-completed
fulfillment-failed
```

Correlation key:

```
fulfillmentCorrelationId = subop_01HZ_KAMAU_RESUME:resume
```

16. Step 12 - Fulfillment Message Correlator Wakes BPMN

FUL-03 publishes or calls back with:

```
{
  "fulfillmentCorrelationId": "subop_01HZ_KAMAU_RESUME:resume",
  "terminalState": "FULLY_ACTIVATED",
  "completedAt": "2026-06-22T10:00:06Z"
}
```

The message correlator calls Camunda:

```
POST /engine-rest/message
Content-Type: application/json

{
  "messageName": "fulfillment-completed",
  "correlationKeys": {
    "fulfillmentCorrelationId": {
      "value": "subop_01HZ_KAMAU_RESUME:resume",
      "type": "String"
    }
  }
}
```

If FUL-03 fails, correlate fulfillment-failed and route to compensation.

17. Step 13 - BPMN Closes Pause History And Writes Audit Fields

Service task topic:

```
sub-lm-put-master-fields
```

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "fieldsJson": {
    "last_status_changed_at": "2026-06-22T10:00:18Z"
  },
  "pauseHistoryCloseOut": {
    "pauseId": "pause_01HZ_KAMAU_SUSPNP",
    "actualResumeAt": "2026-06-22T10:00:18Z",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "resumeActorUserId": "bil04-svc-account",
    "resumeActorRole": "BILLING_INTERNAL",
    "reconnectionFeeBillableEventId": null,
    "adminForceResume": false,
    "adminForceResumeReason": null,
    "dunningCycleReferenceAtResume": "bil04-resume-2026-06-22"
  }
}
```

Implementation guidance:

- resumeReasonCode comes from request/system caller, not from resumeTrigger.
- If SUB-LM and pause-history persistence are in one module, this topic can call one application service that writes both audit fields and pause-history close-out in one transaction.
- If they are split, this topic should call approved internal APIs or a dedicated orchestration service.
- Do not close the pause row before FUL-03 reactivation succeeds.

18. Step 14 - BPMN Commits Subscription State

Service task topic:

sub-lm-commit-state

Worker input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "expectedFromStatus": "PENDING_RESUME",
  "toStatus": "ACTIVE"
}
```

Only after this succeeds can the BPMN emit SubscriptionResumed.

19. Step 15 - BPMN Emits Event

Service task topic:

kafka-emit

Topic:

sophix.subscription.subscription.resumed

Payload:

```
{
  "eventType": "SubscriptionResumed",
  "eventVersion": "1.0",
  "operationId": "subop_01HZ_KAMAU_RESUME",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "resumeTrigger": "DUNNING_RESUME",
  "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
  "closedPauseHistoryRowId": "pause_01HZ_KAMAU_SUSPNP",
  "closedPauseOriginIntent": "SUSPEND_NP",
  "resumedAt": "2026-06-22T10:00:18Z",
  "reconnectionFeeBillableEventId": null
}
```

Write through the outbox. Do not block BPMN completion on Kafka broker availability.

20. Step 16 - BPMN Sends Notification

Service task topic:

notification-send

Worker input:

```
{
  "recipientKind": "customer",
  "recipientRef": "cust_01HZ_KAMAU",
  "notificationEventKind": "SUBSCRIPTION_RESUMED",
  "eventContext": {
    "resumeTrigger": "DUNNING_RESUME",
    "resumeReasonCode": "DEBT_CLEARED_VIA_MPESA",
    "resumedAt": "2026-06-22T10:00:18Z",
    "closedPauseOriginIntent": "SUSPEND_NP",
    "pauseDurationDays": 7,
    "reconnectionFee": null
  }
}
```

If notification fails, log it and continue. Do not compensate a successful service resume because of notification failure.

21. Step 17 - BPMN Finalizes Operation

Service task topic:

subscription-operation-finalize

Worker input:

```
{
  "operationId": "subop_01HZ_KAMAU_RESUME",
}
```

```
    "finalState": "COMPLETED"
  }
}
```

Final API-visible status:

```
{
  "operationId": "subop_01HZ_KAMAU_RESUME",
  "finalState": "COMPLETED",
  "subscriptionStatus": "ACTIVE"
}
```

22. Compensation Flow Details

22.1 Drools Rejection

No mutation should have happened yet.

Flow:

```
kafka-emit SubscriptionResumeRejected
subscription-operation-finalize FAILED
```

22.2 Billing Or Debt Failure

If BIL-01 reports debt still open:

```
sub-lm-revert-to-prior
kafka-emit SubscriptionResumeRejected with DEBT_NOT_CLEARED
subscription-operation-finalize FAILED
```

If a reconnection fee charge exists or payment times out:

```
bil01-revert-pending-charge
sub-lm-revert-to-prior
kafka-emit SubscriptionResumeRejected
subscription-operation-finalize FAILED
```

22.3 Fulfillment Failure

If FUL-03 returns fulfillment - failed:

```
bil01-revert-pending-charge if billableEventId exists
sub-lm-revert-to-prior
kafka-emit SubscriptionResumeRejected
subscription-operation-finalize FAILED
```

22.4 Commit Failure After Network Reactivation

If pause-history close-out or sub-lm-commit-state repeatedly fails, route to manual recovery.

Reason:

The network may already be active. Automatic rollback can be unsafe because the workflow may not know whether downstream reactivation fully completed.

23. Manual Smoke Test - Dunning Resume

1. Seed WIK subscription_operation_config and subscription_resume_config.
2. Deploy Drools package rules.subscription.resume.wik.
3. Deploy BPMN sub-resume-wik.
4. Start worker apps subscribed to required topics.
5. Create or identify a SUSPENDED WIK subscription.
6. Create an open pause-history row with origin_intent = SUSPEND_NP.
7. Send the resume API request with DUNNING_RESUME using BIL-04 service token.
8. Confirm API returns 202 and an operation id.
9. Confirm Camunda process instance key is sub-resume-wik.
10. Confirm subscription status becomes PENDING_RESUME.
11. Confirm BIL-01 receives RESUME_NP_INTENT and returns debtClearedConfirmation=true.

12. Correlate fulfillment-completed.
13. Confirm subscription_pause_history.actual_resume_at is populated.
14. Confirm subscription status becomes ACTIVE.
15. Confirm outbox contains SubscriptionResumed.
16. Confirm operation final state is COMPLETED.

24. Manual Smoke Test - Scheduled Auto Resume

1. Create or identify a SUSPENDED WIK subscription.
2. Create an open pause-history row with:

`origin_intent = CUSTOMER_REQUESTED_PAUSE, resume_scheduled_at <= now, actual_resume_at = NULL.`

3. Send the resume API request with scheduler service token and SCHEDULED_AUTO_RESUME.
4. Confirm Drools accepts scheduler role and scheduled row.
5. Confirm payment gate is skipped.
6. Correlate fulfillment-completed.
7. Confirm pause-history row closes.
8. Confirm subscription becomes ACTIVE.
9. Confirm SubscriptionResumed event has `resumeTrigger = SCHEDULED_AUTO_RESUME`.

25. Common Mistakes

- Treating worker-topic names as shell commands.
- Reading current subscription status from cache.
- Starting BPMN without loading the open pause-history row.
- Writing `resume_reason_code = resumeTrigger`; use `resumeReasonCode`.
- Letting customers self-resume SUSPEND_NP.
- Letting BIL-04 dunning-resume voluntary pauses.
- Removing the payment gateway because WIK default has no reconnection fee.
- Closing pause history before FUL-03 succeeds.
- Emitting SubscriptionResumed before `sub-lm-commit-state`.
- Failing the whole operation because NOT-01 notification failed.